ASSESSMENT - 1

# Scalability Audit of an E-Commerce Platform (ShopSphere)

IT Application-Based Problem Solving

**NAME:** Nabilah Thahseen .B
**REG NO:** 1925260057
**COURSE CODE / NAME:** CSA0610 - Design and Analysis of Algorithms

A detailed algorithm analysis report covering operation counts, recurrence relations, asymptotic bounds, search strategies, and scalability recommendations.

# Contents


| Section | Description |
|---|---|
| 1 | Introduction, objectives and assumptions |
| 2 | Task (a): Understand the problem and identify basic operations |
| 3 | Task (b): Iterative modules M1 and M2 |
| 4 | Task (c): Binary search recurrence by backward substitution |
| 5 | Task (d): Master Theorem and recursion-tree analysis |
| 6 | Task (e): Best, average and worst cases of S1 and S2 |
| 7 | Task (f): Proving the asymptotic bound |
| 8 | Task (g): Scalability decision and optimisation plan |
| 9 | Testing strategy, limitations and final conclusion |
| 10 | Appendix: Python verification program |


# 1. Introduction

ShopSphere is an e-commerce platform whose product catalogue is expected to grow from 100,000 products to 1,000,000 products within a year. This tenfold increase can make algorithms that currently seem acceptable become slow or impractical. A scalability audit estimates how the work performed by each algorithm changes as the input grows.

This report studies five backend modules and two product-search strategies. It identifies each basic operation, counts operations for iterative algorithms, solves recursive relations, compares search cases, proves a tight asymptotic bound, and evaluates whether the modules can fit into a restricted nightly processing window.

## 1.1 Objectives

- Identify the input size and dominant operation for each module.
- Derive exact or formula-based operation counts where possible.
- Use Big-O, Big-Omega and Big-Theta notation correctly.
- Compare sequential search with binary search.
- Apply backward substitution, the Master Theorem and recursion-tree reasoning.
- Estimate processing time using the assignment's stated operation rate.
- Recommend realistic improvements without changing required behaviour.

### 1.2 Assumptions and notation

| Item | Assumption |
|---|---|
| Input size n | Number of products, requests or sales records, depending on the module. |
| Logarithms | Base 2 unless stated otherwise. |
| Processing rate | 10 basic operations per second. |
| Processing window | 10 minutes = 600 seconds. |
| Operation budget | 10 x 600 = 6,000 basic operations. |
| Measurement type | Theoretical operation-count estimate, not a benchmark of a real server. |

**A key distinction is that asymptotic complexity describes growth, while the processing-window calculation uses the specific constants and operation counts assumed in this assignment.**

## 2. Task (a) - Understand the problem

The first step is to define what one unit of work means in each module. Counting a consistent basic operation makes the comparison meaningful.

| Module / strategy | Basic operation | Growth rate | Initial assessment |
|---|---|---|---|
| M1 - Duplicate detector | Compare title and seller for a pair of products | Theta(n^2) | Poor for very large catalogues |
| M2 - API rate limiter | Execute Audit(Request[j]) | Theta(n) | Linear, but the given count can exceed the budget |
| S1 - Sequential search | Compare key with current product | Theta(n) worst case | Simple; scans one by one |
| S2 - Binary search | Compare key with middle indexed entry | Theta(log n) | Very efficient when index is sorted |
| M4 - Current recommender | Work across four half-size recursive calls | Theta(n^2) | Expensive at scale |
| M4 - Redesigned recommender | Work across two half-size calls plus linear merge work | Theta(n log n) | Improved growth |
| M5 - Sales analytics | Sort-related work at recursive levels | Theta(n log^2 n) | More costly than n log n |
| Report generator | Dominant quadratic calculation | Theta(n^2) | Quadratic term dominates |

### 2.1 Why the growth rate matters

If n increases by a factor of ten, a linear algorithm typically does about ten times as much work, while a quadratic algorithm does about one hundred times as much. A logarithmic algorithm grows much more slowly. This is why algorithm design is central to platform scalability.

- Linear work is often suitable for a single pass, although strict operation budgets may still be exceeded.
- Quadratic work is a warning sign when processing hundreds of thousands or millions of records.
- Logarithmic search is attractive for repeated lookups, provided the data structure supports it.
- Better asymptotic growth does not automatically mean the module meets every fixed deadline.

# 3. Task (b) - Analyse the iterative modules

## 3.1 M1 - Duplicate-listing detector

The duplicate detector compares each product with the products after it. For the first product there are n - 1 comparisons; for the next there are n - 2, continuing until one comparison remains.

C(n) = (n - 1) + (n - 2) + ... + 2 + 1

Using the sum of the first n - 1 positive integers:

C(n) = n(n - 1) / 2 = (n^2 - n) / 2

The dominant term is n^2 / 2, so the time complexity is Theta(n^2). The algorithm uses a fixed number of loop variables, giving Theta(1) auxiliary space when the input list is not counted.

| Number of products | Comparisons |
|---|---|
| 1,000 | 499,500 |
| 100,000 | 4,999,950,000 |
| 1,000,000 | 499,999,500,000 |

The catalogue growing from 100,000 to 1,000,000 products increases this comparison count by approximately 100 times. Under the stated model, this approach is not scalable.

### 3.1.1 Recommended redesign

Use a hash set keyed by the pair (title, seller). For each product, check whether the key is already present; if it is, record a duplicate, otherwise insert the key. Expected processing time becomes Theta(n), with Theta(n) additional memory. If hash-based storage is unsuitable, sorting by the same key and comparing adjacent products gives an O(n log n) alternative.

## 3.2 M2 - API rate limiter

The outer loop starts with i = 1 and doubles i after each pass. The inner loop executes i audit operations. The work therefore forms a geometric series:

A(n) = 1 + 2 + 4 + 8 + ... + 2^(k-1)

For k passes, the sum is 2^k - 1. With the loop condition i < n, the exact count for this implementation is the sum of powers of two strictly below n. This is less than 2n and therefore Theta(n).

| n | Audit operations | Outer-loop passes |
|---|---|---|
| 1,024 | 1,023 | 10 |
| 100,000 | 131,071 | 17 |
| 1,000,000 | 1,048,575 | 20 |

The number of outer-loop passes is logarithmic, but the total inner-loop work is linear because the inner-loop sizes form a geometric series. Therefore, the total is not Theta(n log n).

### 3.2.1 Operational considerations

- Process requests in bounded batches so a large audit does not delay other backend work.
- Retain rate-limit correctness and security checks when reducing audit work.
- Consider partitioning records across workers if the workload can be safely parallelised.

- Measure actual request volume and audit cost before changing production policy.

# 4. Task (c) - Solve binary search by backward substitution

Binary search works on a sorted index. Each comparison examines the middle entry and discards approximately half of the remaining search range. Its comparison recurrence is:

T(n) = T(n/2) + 1, with T(1) = 1

## 4.1 Backward substitution

Expand the recurrence repeatedly:

T(n) = T(n/2) + 1

= T(n/4) + 2

= T(n/8) + 3

After i expansions, T(n) = T(n / 2^i) + i. The base case is reached when n / 2^i = 1, which means i = log2(n).

For power-of-two input sizes, T(n) = 1 + log2(n). For arbitrary integer n, the worst-case comparison count is floor(log2(n)) + 1.

| Input size | Worst-case comparisons |
|---|---|
| 1,024 | 11 |
| 1,000,000 | 20 |

Binary search is Theta(log n) in time. Iterative binary search uses Theta(1) auxiliary space; a recursive implementation uses Theta(log n) call-stack space.

At one million entries, 20 comparisons divided by 10 operations per second gives an estimated 2 seconds in this simplified model. This is well within the 600-second window.

## 4.2 Preconditions and practical note

- The list or index must be sorted according to the same ordering used by the search.
- Updates to products must preserve or rebuild the sorted index.
- Binary search is excellent for lookup, but index construction and maintenance have costs of their own.
- Duplicate keys require a defined policy, such as returning any match or finding the first matching entry.

# 5. Task (d) - Master Theorem and recursion trees

The Master Theorem is commonly applied to recurrences of the form T(n) = aT(n/b) + f(n), where a is the number of recursive subproblems, n/b is the size of each subproblem, and f(n) is the work outside recursive calls.

## 5.1 M4 - Current recommendation engine

The current design is T(n) = 4T(n/2) + n. Here a = 4, b = 2, and f(n) = n. Since n^(log base 2 of 4) = n^2 and f(n) grows more slowly than n^2, Master Theorem Case 1 applies.

Therefore, T(n) = Theta(n^2). For powers of two with base value T(1) = 1, the exact recurrence solution is T(n) = 2n^2 - n.

### 5.2 M4 - Proposed redesign

The redesigned recurrence is T(n) = 2T(n/2) + n. Now a = 2, b = 2, and n^(log base 2 of 2) = n. Since f(n) = n has the same order as n, Master Theorem Case 2 applies.

Therefore, T(n) = Theta(n log n). For powers of two and T(1) = 1, the exact solution is T(n) = n log2(n) + n.

| Input n | Current design: 2n^2 - n | Redesigned: n log2(n) + n |
|---|---|---|
| 1,024 | 2,096,128 | 11,264 |
| 1,000,000 (formula estimate) | 1,999,999,000,000 | about 20,931,569 |

The redesigned algorithm greatly reduces the growth rate. However, around 20.9 million operations is still above the stated budget of 6,000 operations. The redesign is a major improvement, but additional optimisation or a different processing assumption is needed.

### 5.3 M5 - Sales analytics engine

The recurrence is T(n) = 2T(n/2) + n log2(n). Here a = 2 and b = 2, so n^(log base 2 of 2) = n. The non-recursive work is n log n, which matches the extended Master Theorem form n^(log_b(a)) log^k(n) with k = 1.

The extended result is T(n) = Theta(n log^2(n)).

#### 5.3.1 Recursion-tree explanation

Let L = log2(n). At level i, there are 2^i subproblems, each of size n / 2^i. The work at one node is (n / 2^i) log2(n / 2^i), so the total work at that level is n(L - i). Summing across internal levels gives n times the sum L + (L - 1) + ... + 1, plus leaf work.

For powers of two and T(1) = 1, the resulting expression is T(n) = n L(L + 1)/2 + n, where L = log2(n).

At n = 1,024, L = 10, so the formula gives 1,024 x 10 x 11 / 2 + 1,024 = 57,344 operations. The complexity grows faster than n log n and is not suitable for a strict 6,000-operation budget at large n.

### 5.4 Recurrence comparison

| Algorithm | Recurrence | Complexity | Interpretation |
|---|---|---|---|
| M4 current | 4T(n/2) + n | Theta(n^2) | Four branches make work grow quickly. |
| M4 redesigned | 2T(n/2) + n | Theta(n log n) | Fewer branches reduce growth. |
| M5 analytics | 2T(n/2) + n log n | Theta(n log^2 n) | Extra logarithmic work at each level increases total work. |

## 6. Task (e) - Best, average and worst cases

### 6.1 S1 - Sequential search

Sequential search checks entries from the beginning until it finds the target or exhausts the list.

- Best case: the target is first, so one comparison is required: Theta(1).
- Worst case: the target is last or absent, so up to n comparisons are required: Theta(n).
- Average successful case: if every position is equally likely, the average is (1 + 2 + ... + n) / n = (n + 1) / 2.

If p is the probability that a search succeeds, and a failed search checks all n entries, the average count is p(n + 1)/2 + (1 - p)n. For n = 1,000,000 and p = 0.5, this is 750,000.25 comparisons.

### 6.2 S2 - Binary search

- Best case: the target is the first middle entry checked, requiring one comparison: Theta(1).
- Worst case: the search continues until the remaining range is empty or the target is found at the deepest level, requiring floor(log2(n)) + 1 comparisons.
- Average case: depends on the shape of the search tree and the distribution of target keys; it is logarithmic for a balanced index.

At one million entries, binary search needs at most 20 comparisons under the stated comparison-count convention. Its average is also around logarithmic scale, unlike sequential search's hundreds of thousands of comparisons under the example assumptions.

| Case at n = 1,000,000 | S1: Sequential | S2: Binary |
|---|---|---|
| Best | 1 comparison | 1 comparison |
| Average (p = 0.5; successful positions uniform) | 750,000.25 comparisons | Approximately 20 or fewer; depends on key distribution |
| Worst | 1,000,000 comparisons | 20 comparisons |
| Time complexity | Theta(n) worst case | Theta(log n) worst case |

Recommendation: use binary search for repeated product lookups when the catalogue can be kept sorted. Include index update and maintenance costs in the broader system design.

## 7. Task (f) - Prove an asymptotic bound

The report generator has the function f(n) = 3n^2 + 10n log2(n) + 50. We want to show f(n) = Theta(n^2). By definition, we need positive constants c1, c2 and n0 such that c1 n^2 <= f(n) <= c2 n^2 for all n >= n0.

### 7.1 Lower bound

For n >= 1, the terms 10n log2(n) and 50 are non-negative. Therefore f(n) >= 3n^2. Choose c1 = 3. This proves f(n) = Omega(n^2).

### 7.2 Upper bound

For n >= 1, log2(n) <= n. Hence 10n log2(n) <= 10n^2. Also, 50 <= 50n^2. Combining these inequalities gives:

f(n) <= 3n^2 + 10n^2 + 50n^2 = 63n^2

Choose c2 = 63 and n0 = 1. Therefore, 3n^2 <= f(n) <= 63n^2 for all n >= 1, which proves f(n) = Theta(n^2).

### 7.3 Why the function is not O(n)

Suppose f(n) were O(n). Then for some constant c and sufficiently large n, f(n) <= cn. But f(n) >= 3n^2, so 3n^2 <= cn and 3n <= c. This cannot remain true as n grows without bound. Therefore f(n) is not O(n).

The quadratic term dominates for large input sizes, which explains why the report generator requires redesign.

| n | f(n) = 3n^2 + 10n log2(n) + 50 | Lower bound 3n^2 | Upper bound 63n^2 |
|---|---|---|---|
| 1 | 53 | 3 | 63 |
| 10 | about 682.19 | 300 | 6,300 |
| 100 | about 36,693.86 | 30,000 | 630,000 |
| 1,000 | about 3,099,707.84 | 3,000,000 | 63,000,000 |

# 8. Task (g) - Scalability decision

The processing rate is 10 basic operations per second and the available window is 600 seconds. The maximum operation count is 10 x 600 = 6,000. Estimated runtime equals operation count divided by 10.

| Module | Operations at n = 1,000,000 (approx.) | Estimated time | Within 10 minutes? |
|---|---|---|---|
| M1 - Duplicate detector | 499,999,500,000 | 49,999,950,000 seconds | No |
| M2 - API rate limiter | 1,048,575 | 104,857.5 seconds | No |
| S2 - Binary search | 20 | 2 seconds | Yes |
| M4 - Current design | 1,999,999,000,000 | 199,999,900,000 seconds | No |
| M4 - Redesigned | 20,931,569 | 2,093,157 seconds | No |
| M5 - Sales analytics | about 209,599,497 | about 20,959,950 seconds | No |
| Report generator | about 3,000,199,315,736 | about 300,019,931,574 seconds | No |

**These times are arithmetic estimates under the assignment's simplified operation-rate assumption. They should not be interpreted as measured production runtimes. Real systems have different operation costs, caching, I/O, concurrency and hardware behaviour.**

## 8.1 Recommended action plan

| Priority | Module | Action | Expected benefit |
|---|---|---|---|
| 1 | M1 duplicate detector | Replace pairwise checks with a hash set or sort-and-scan | Expected Theta(n) or Theta(n log n) work |
| 2 | Report generator | Remove repeated nested scans; aggregate in one pass where possible | Reduce dominant quadratic work |
| 3 | M4 recommender | Adopt the two-branch redesign and profile expensive repeated calculations | Change growth from Theta(n^2) to Theta(n log n) |
| 4 | M5 analytics | Investigate whether sorting can be performed once and merged | Potentially reduce Theta(n log^2 n) to Theta(n log n) |
| 5 | M2 rate limiter | Batch or distribute audits while preserving security requirements | Improve operational scheduling |
| 6 | S2 binary search | Maintain a sorted index and test update consistency | Keep lookups efficient |

## 8.2 How to prioritise improvements

First address algorithms whose work is quadratic and whose output requirements permit a more efficient data structure. Then examine modules with n log^2 n growth. The linear rate limiter should be reviewed against the actual workload and deadline, even though its asymptotic class is reasonable. Binary search already performs well, so the main responsibility is maintaining its sorted-index precondition.

- Correctness: ensure the improved algorithm returns the same required result.
- Performance: measure operation counts and runtime over several input sizes.
- Memory: account for extra storage required by hash tables and indexes.
- Reliability: test duplicates, missing keys, empty input and boundary values.
- Maintainability: document assumptions and keep complexity comments close to the implementation.

# 9. Testing strategy, limitations and conclusion

## 9.1 Suggested verification plan

- Test small inputs where the expected count can be checked by hand.
- Test power-of-two inputs for recurrence formulas, such as n = 1,024.
- Test non-power-of-two inputs to confirm rounding and loop termination.
- Compare iterative and recursive versions on the same small input.
- Check the asymptotic-bound inequalities at several values of n.
- Record measured runtime separately from theoretical operation-count estimates.

## 9.2 Limitations

The report uses the stated basic-operation rate of 10 operations per second, which is an artificial model for comparing algorithms. A real backend includes database access, network delay, memory allocation, cache behaviour and concurrent requests. Some exact recurrence formulas also assume power-of-two input sizes and a specific base case. Practical deployment decisions should be validated through profiling and load testing.

## 9.3 Conclusion

The scalability audit shows that algorithm growth rate is essential when an e-commerce catalogue expands. M1 and the report generator are quadratic and should be redesigned first. The current M4 recurrence is also quadratic; the proposed redesign improves it to Theta(n log n). M5 has Theta(n log^2 n) complexity, while M2 is linear. S2, binary search, is the most efficient search strategy considered and remains within the simplified processing window.

Under the assignment's stated rate and deadline, only S2 meets the 6,000-operation budget at one million records. The other modules need further optimisation, batching, workload distribution, or a reassessment of the simplified operation model. The final decision should combine mathematical analysis with correctness tests and measured performance.

# 10. Appendix - Python verification program

The following compact program verifies several important formulas used in the report. It avoids third-party packages and can be run with a standard Python 3 installation.

```
import math

def duplicate_comparisons(n):
    return n * (n - 1) // 2

def rate_limiter_audits(n):
    i = 1
    total = 0
    while i < n:
        total += i
        i *= 2
    return total

def binary_search_worst(n):
    return math.floor(math.log2(n)) + 1

def report_work(n):
    return 3 * n**2 + 10 * n * math.log2(n) + 50

def main():
    print("DUPLICATE DETECTOR")
    for n in (1000, 100000, 1000000):
        print(n, duplicate_comparisons(n))

    print("\nAPI RATE LIMITER")
    for n in (1024, 100000, 1000000):
```

```
        print(n, rate_limiter_audits(n))

    print("\nBINARY SEARCH")
    for n in (1024, 1000000):
        print(n, binary_search_worst(n))

    print("\nREPORT GENERATOR BOUND")
    for n in (1, 10, 100, 1000):
        value = report_work(n)
        print(n, round(value, 2),
              "lower:", 3*n**2,
              "upper:", 63*n**2,
              "holds:", 3*n**2 <= value <= 63*n**2)

    print("\nPROCESSING WINDOW")
    rate = 10
    window = 600
    print("Maximum operations:", rate * window)
    print("Binary search time:",
          binary_search_worst(1000000) / rate, "seconds")

if __name__ == "__main__":
    main()
```

### Expected output (main values)

| Check | Expected value |
|---|---|
| M1 at n = 1,000 | 499,500 comparisons |
| M1 at n = 100,000 | 4,999,950,000 comparisons |
| M2 at n = 1,024 | 1,023 audits |
| M2 at n = 1,000,000 | 1,048,575 audits |
| S2 at n = 1,024 | 11 comparisons |
| S2 at n = 1,000,000 | 20 comparisons |
| Processing budget | 6,000 operations |

# Code of Conduct

I, Nabilah Thahseen .B (Reg. No. 1925260057), certify that this submission has been prepared for academic learning and that sources or reference material used for guidance should be acknowledged in accordance with the institution's academic integrity requirements.

Signature of the Student: _____

Date: _____